

LibTracker - Bug Report & Fixes

9 issues found and resolved | 2026-04-27

Summary of All Issues

#	Title	File	Severity
1	Timezone-naive vs aware datetime comparison crash	backend/routes/reservations.py	CRITICAL
2	int() conversion without error handling in create_library	backend/routes/libraries.py	CRITICAL
3	int() conversion without error handling in update_library	backend/routes/libraries.py	CRITICAL
4	Admin email is not a valid email address	backend/database.py	LOGICAL BUG
5	Seed generates future timestamps for UsageStatistics	backend/database.py	LOGICAL BUG
6	Capacity update resets available_seats ignoring active reserva	backend/routes/libraries.py	LOGICAL BUG
7	current_occupancy can exceed total_capacity	backend/routes/libraries.py	LOGICAL BUG
8	available_seats counter becomes permanently inaccurate	backend/routes/reservations.py	LOGICAL BUG
9	N+1 query: one DB call per feedback row	backend/routes/feedback.py	PERFORMANCE

LibTracker - Bug Report & Fixes

9 issues found and resolved | 2026-04-27

#1 Timezone-naive vs aware datetime comparison crash

Severity: **CRITICAL**
Location: backend/routes/reservations.py line 83

Description
Comparing a timezone-aware datetime (e.g. sent as ISO 8601 with +03:00) against naive datetime.now() raises TypeError at runtime. The fix uses the incoming datetime's own tzinfo, falling back to UTC.

Before (buggy code):
`if start < datetime.now():`

After (fixed code):
`if start < datetime.now(start.tzinfo or timezone.utc):`

LibTracker - Bug Report & Fixes

9 issues found and resolved | 2026-04-27

#2 int() conversion without error handling in create_library

Severity: **CRITICAL**

Location: backend/routes/libraries.py line 87

Description

Passing a non-numeric string (e.g. "abc") causes an unhandled ValueError and returns a 500 Internal Server Error instead of a 400. Wrapped in try/except so invalid input is rejected cleanly.

Before (buggy code):

```
capacity = int(data.get("total_capacity", 0))
```

After (fixed code):

```
try:
    capacity = int(data.get("total_capacity", 0))
except (ValueError, TypeError):
    return jsonify({"error": "total_capacity must be a valid number"}), 400
```

LibTracker - Bug Report & Fixes

9 issues found and resolved | 2026-04-27

#3 int() conversion without error handling in update_library

Severity: **CRITICAL**

Location: backend/routes/libraries.py lines 134, 136

Description

Same crash as bug #2 but in the update endpoint. Both fields are now validated with try/except before assignment.

Before (buggy code):

```
lib.total_capacity = int(data["total_capacity"])
lib.current_occupancy = int(data["current_occupancy"])
```

After (fixed code):

```
try:
    lib.total_capacity = int(data["total_capacity"])
except (ValueError, TypeError):
    return jsonify({"error": "total_capacity must be a valid number"}), 400

try:
    new_occ = int(data["current_occupancy"])
except (ValueError, TypeError):
    return jsonify({"error": "current_occupancy must be a valid number"}), 400
```

#4 Admin email is not a valid email address

Severity: **LOGICAL BUG**
Location: backend/database.py line 62

Description
The seed creates an admin user with the email string "admin", which is not a valid email format. Changed to admin@libtracker.edu so the field holds a properly formed address.

Before (buggy code):
admin_email = "admin"

After (fixed code):
admin_email = "admin@libtracker.edu"

LibTracker - Bug Report & Fixes

9 issues found and resolved | 2026-04-27

#5 Seed generates future timestamps for UsageStatistics

Severity: **LOGICAL BUG**

Location: backend/database.py line 128

Description

When `now.hour < hour` (e.g. it is 9 AM and `hour=14`), the subtraction becomes negative, effectively adding hours and creating a future timestamp. Fixed by using `.replace()` to set the exact hour on the past date, then skipping any slot still in the future.

Before (buggy code):

```
recorded = now - timedelta(days=day_offset, hours=(now.hour - hour))
```

After (fixed code):

```
recorded = (now - timedelta(days=day_offset)).replace(
    hour=hour, minute=0, second=0, microsecond=0
)
if recorded > now:
    continue
```

#6 Capacity update resets available_seats ignoring active reservations

Severity: LOGICAL BUG
Location: backend/routes/libraries.py lines 142-146

Description
When an admin changes a library's total capacity, all study areas had their available_seats reset to the new per-area value, ignoring existing active reservations. A fully booked area would suddenly appear empty. Fixed by computing available seats from the real active reservation count.

Before (buggy code):
area.total_seats = new_seats_per_area
area.available_seats = new_seats_per_area # ignores reservations

After (fixed code):
active_count = Reservation.query.filter_by(
 study_area_id=area.id, status="active"
) .count()
area.total_seats = new_seats_per_area
area.available_seats = max(0, new_seats_per_area - active_count)

LibTracker - Bug Report & Fixes

9 issues found and resolved | 2026-04-27

#7 current_occupancy can exceed total_capacity

Severity: **LOGICAL BUG**

Location: backend/routes/libraries.py line 136

Description

No upper-bound check allowed an admin to set current_occupancy to any value, making occupancy_percentage return values above 100%. Added validation against the already-updated total_capacity.

Before (buggy code):

```
lib.current_occupancy = int(data["current_occupancy"])
```

After (fixed code):

```
if new_occ > lib.total_capacity:  
    return jsonify({"error": "current_occupancy cannot exceed capacity"}), 400  
lib.current_occupancy = new_occ
```


LibTracker - Bug Report & Fixes

9 issues found and resolved | 2026-04-27

#8 available_seats counter becomes permanently inaccurate

Severity: LOGICAL BUG
Location: backend/routes/reservations.py lines 103-107

Description
When available_seats reached 0, the counter stopped decrementing while new reservations were still being created. After cancellations restored the counter, subsequent bookings left it out of sync. Fixed by recomputing available_seats from actual active reservation count. The same fix was applied to cancel_reservation.

Before (buggy code):

```
if area.available_seats > 0:
    area.available_seats -= 1
```

After (fixed code):

```
db.session.flush()
active_count = Reservation.query.filter_by(
    study_area_id=study_area_id, status="active"
).count()
area.available_seats = max(0, area.total_seats - active_count)
```

LibTracker - Bug Report & Fixes

9 issues found and resolved | 2026-04-27

#9 N+1 query: one DB call per feedback row

Severity: **PERFORMANCE**

Location: backend/routes/feedback.py lines 126-129

Description

With the default limit of 20 feedbacks, this loop fired 21 separate SQL queries (one per row plus one for the list). Replaced with a single JOIN using SQLAlchemy's `joinedload()`, reducing it to 1 query regardless of the number of results.

Before (buggy code):

```
for fb in feedbacks:
    user = User.query.get(fb.user_id) # 1 query per row
```

After (fixed code):

```
feedbacks = (
    Feedback.query
    .options(joinedload(Feedback.user))
    ...
)
for fb in feedbacks:
    d["user_name"] = fb.user.name if fb.user else "Anonim"
```